Exercise Sheet 6: Generative modeling with Normalizing Flows

Due on 12.06.2026, 10:00

Ulrich Prestel (U.Prestel@lmu.de)

Important Notes:

1. **Questions:** Please use the Moodle platform and post your questions to the forum. They will be answered by us or your fellow students.
2. **Deadline:** The due date for this exercise is **12.06.2026, 10:00**.
3. **Submission:** Your submission should consist of one single ZIP file. Both the PDF file and the ZIP file should contain your surname and your matriculation number (`Surname-MatriculationNumber.zip`) for grading purposes. Please submit the completed `.ipynb` notebook (the provided template, with your code and answers filled in) together with an exported PDF version of the notebook serving as the report. Submissions that fail to follow the naming convention or miss the PDF/notebook files will not be graded.

1 Normalizing flows with RealNVP

Background. A normalizing flow is an invertible neural network

$$z = f_\theta(x) \tag{1}$$

that maps data points $x \in \mathbb{R}^D$ to latent codes $z \in \mathbb{R}^D$. We choose the latent distribution as $p_Z(z) = \mathcal{N}(0, I)$. By the change-of-variables formula,

$$\log p_X(x) = \log p_Z(f_\theta(x)) + \log \left| \det J_{f_\theta}(x) \right|, \tag{2}$$

where

$$J_{f_\theta}(x) = \frac{\partial f_\theta(x)}{\partial x} \tag{3}$$

is the Jacobian of the full flow. Thus, the model is trained by minimizing the negative log-likelihood. A common architecture for normalizing flows, called *RealNVP*[1], is a composition of invertible *affine coupling layers*

$$f_\theta = f_N \circ f_{N-1} \circ \cdots \circ f_1. \tag{4}$$

[1] https://arxiv.org/pdf/1605.08803

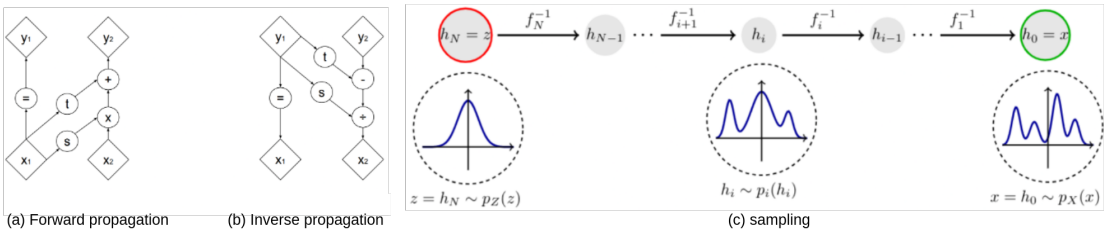

Figure 1: **Anatomy of a normalizing flow**. (a): forward propagation. (b): inverse propagation. (c): stacking affine coupling layers. The inverse flow transforms samples from the simple distribution p_Z to the data distribution.

Affine Coupling Layers. Each affine coupling layer f_i maps an input vector $x \in \mathbb{R}^D$ to an output vector $y \in \mathbb{R}^D$. First, we split the input, which we denote by

$$x = (x_1, x_2), \qquad x_1 \in \mathbb{R}^{D_1}, \quad x_2 \in \mathbb{R}^{D_2}, \tag{5}$$

with $D_1 + D_2 = D$. For simplicity, we only use even input dimensions in this exercise and split the input in half, i.e. $D_1 = D_2 = D/2$. The affine coupling layer is defined as

$$f_i(x) = y \quad \Longleftrightarrow \quad \begin{cases} x = (x_1, x_2), \\ y_1 = x_1, \\ y_2 = x_2 \odot \exp\big(s_i(x_1)\big) + t_i(x_1), \\ y = \text{concat}(y_1, y_2), \end{cases} \tag{6}$$

where $s_i, t_i : \mathbb{R}^{D_1} \to \mathbb{R}^{D_2}$ are small neural networks and $\odot$ denotes element-wise multiplication. The inverse transformation is available in closed form:

$$f_i^{-1}(y) = x \quad \Longleftrightarrow \quad \begin{cases} y = (y_1, y_2), \\ x_1 = y_1, \\ x_2 = \big(y_2 - t_i(y_1)\big) \odot \exp\big(-s_i(y_1)\big), \\ x = \text{concat}(x_1, x_2). \end{cases} \tag{7}$$

The coupling is visualized[2] in Figure 1 . Between coupling layers, we use a swap operation

$$\text{swap}\big((x_1, x_2)\big) = (x_2, x_1). \tag{8}$$

This ensures that both halves of the input are transformed across multiple layers.

[2]The exp operation is omitted in Figure 1 for visual clarity.

Training. For one coupling layer f_i, define its Jacobian as

$$J_i(x) = \frac{\partial f_i(x)}{\partial x}. \tag{9}$$

The Jacobian of a coupling layer is triangular. Therefore,

$$\log \left| \det J_i(x) \right| = \sum_{k=1}^{D_2} s_{i,k}(x_1). \tag{10}$$

For the full RealNVP, the log-determinants add [3]: Let $h_0 = x$ and let h_{i-1} denote the input to the i-th coupling layer. Then

$$\log \left| \det J_{f_\theta}(x) \right| = \sum_{i=1}^{N} \log \left| \det J_i(h_{i-1}) \right|. \tag{11}$$

For a batch of B data points $x^{(1)}, \ldots, x^{(B)}$, compute

$$z^{(m)} = f_\theta(x^{(m)}), \qquad m = 1, \ldots, B, \tag{12}$$

and the corresponding log-determinants $\log \left| \det J_{f_\theta}(x^{(m)}) \right|$. Since $p_Z(z) = \mathcal{N}(0, I)$, the loss can be implemented as

$$\mathcal{L} = -\frac{1}{B} \sum_{m=1}^{B} \left(-\frac{1}{2} \sum_{k=1}^{D} \left(z_k^{(m)} \right)^2 + \log \left| \det J_{f_\theta}(x^{(m)}) \right| \right), \tag{13}$$

where the additive Gaussian normalization constant is omitted.

Sampling. To generate samples, draw $z \sim \mathcal{N}(0, I)$ and apply the inverse model:

$$x = f_\theta^{-1}(z). \tag{14}$$

Task 1: Two Moons with RealNVP **(10P)**

Implement a RealNVP model and train it on the two moons dataset `sklearn.datasets.make_moons(noise=0.1)`. Since we are working in 2D, we have $D = 2$. In `ex06.ipynb` there is a rough skeleton, where you should fill in the gaps for:

- affine coupling layers,

[3] The swap is invertible and has absolute determinant one, so it does not change the log-determinant.

- a forward pass $x \mapsto z$,
- an inverse pass $z \mapsto x$,
- the accumulated log-determinant,
- a function `sample(num_samples)`.

Use two-layer MLPs with ReLU activations for s_θ and t_θ. For stability, let the MLP output $\tilde{s}$ and use the log-scale $s = \tanh(\tilde{s})$. In the implementation below, swap operations are inserted between consecutive coupling layers; we omit them in Eq. (4) for readability. First verify numerically that

$$f_\theta^{-1}(f_\theta(x)) \approx x. \tag{15}$$

Then train the model with the negative log-likelihood loss.
Show:

- samples from the training data,
- generated samples from the RealNVP,
- latent codes $z = f_\theta(x)$ for test data.

Briefly comment on the quality of the generated samples and on whether the latent codes look approximately normally distributed. Train the network for 100 epochs with learning rate $1e-3$. Repeat the experiment for different hidden dimensions for s, t for hidden dimensions $[16, 64, 128]$ as well as the number of coupling blocks $[2, 5, 10]$.

Task 2: Digits with RealNVP (5P)

Repeat the experiment with the digits dataset `sklearn.datasets.load_digits()`. Flatten each 8×8 image into a vector $x \in \mathbb{R}^{64}$ and train a RealNVP with input dimension $D = 64$. Use hidden dimension 128, and 8 coupling blocks, and train for 2000 epochs.
Show:

- real digit images,
- generated digit images,

Try to overfit on a single digit (for example 5) and compare the results with the model trained on all digits.

Task 3: Conditional RealNVP on digits (5P)

Extend your RealNVP implementation to a conditional RealNVP for

$$p(x \mid y), \tag{16}$$

where y is the digit label. The coupling blocks and training loss work analogously in this setting. Use a one-hot encoding c of the label and pass it as an additional input to the coupling networks s, t:

$$y_1 = x_1, \tag{17}$$
$$y_2 = x_2 \odot \exp(s_\theta(x_1, c)) + t_\theta(x_1, c). \tag{18}$$

Train the conditional RealNVP on the digits dataset. Generate samples for fixed labels $0, \ldots, 9$ and show whether the generated images correspond to the requested labels. Use hidden dimension 512. How does this model compare to the model from the previous task?